

CST-407 Milestone 3 Implement Data Sanitization to Prevent XSS Attacks

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

09/3/2025

Link to Screencast: [CST407 - Milestone 3 - Implement Data Sanitization to Prevent XSS Attacks](#)

Table of Contents

Step 1 — Identify XSS Vulnerable Points.....	3
1.1 Audit Method and Findings	3
1.2 Proof-of-Concept Attack (repro steps)	4
Create Event With Payload	4
XSS Popup On Events Page.....	5
Step 2 — Implement Data Sanitization.....	6
2.1 Fix the template sink (escape on render).....	6
2.2 Add a sanitization library (jsoup).....	7
2.3 Create a sanitizer component.....	8
2.4 Sanitize before saving	9
EventService Injection	9
EventService Save Sanitizes.....	10
2.5 Rebuild & run.....	11
Step 3 — Test the Application	12
3.1 Test Plan	12
3.2 Execute the Tests	13
Malicious Script	13
Malicious Image Event	14
Normal Text Event.....	15
Mixed Input	16
3.3 Final Validation	17
Summary of Key Points	18

Step 1 — Identify XSS Vulnerable Points

1.1 Audit Method and Findings

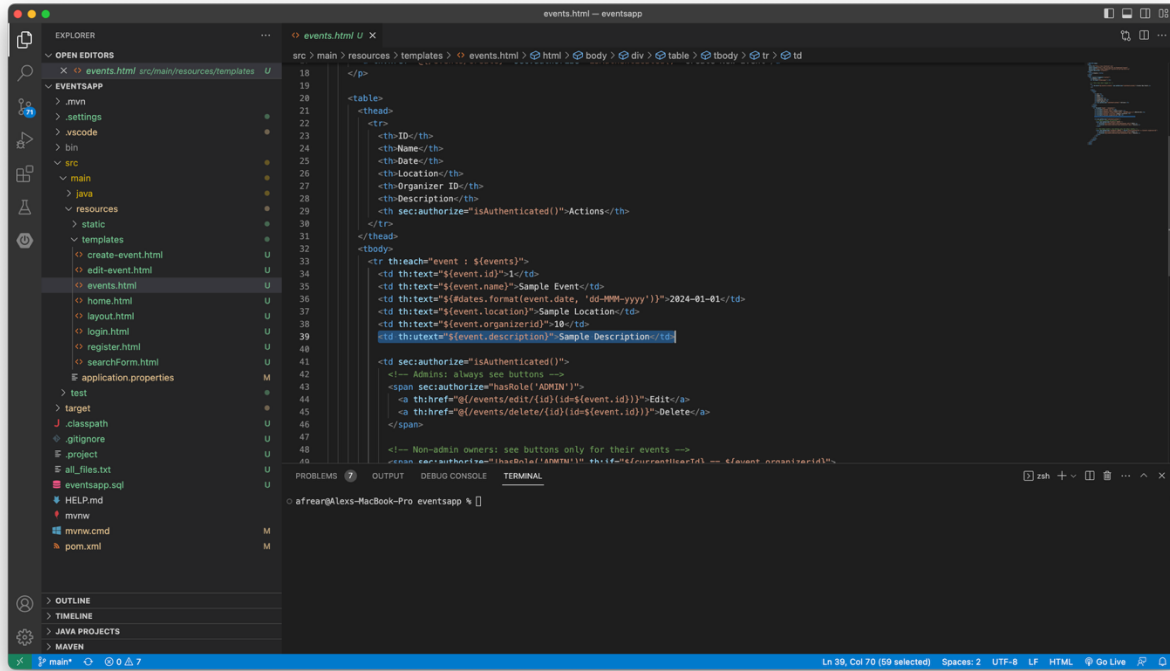
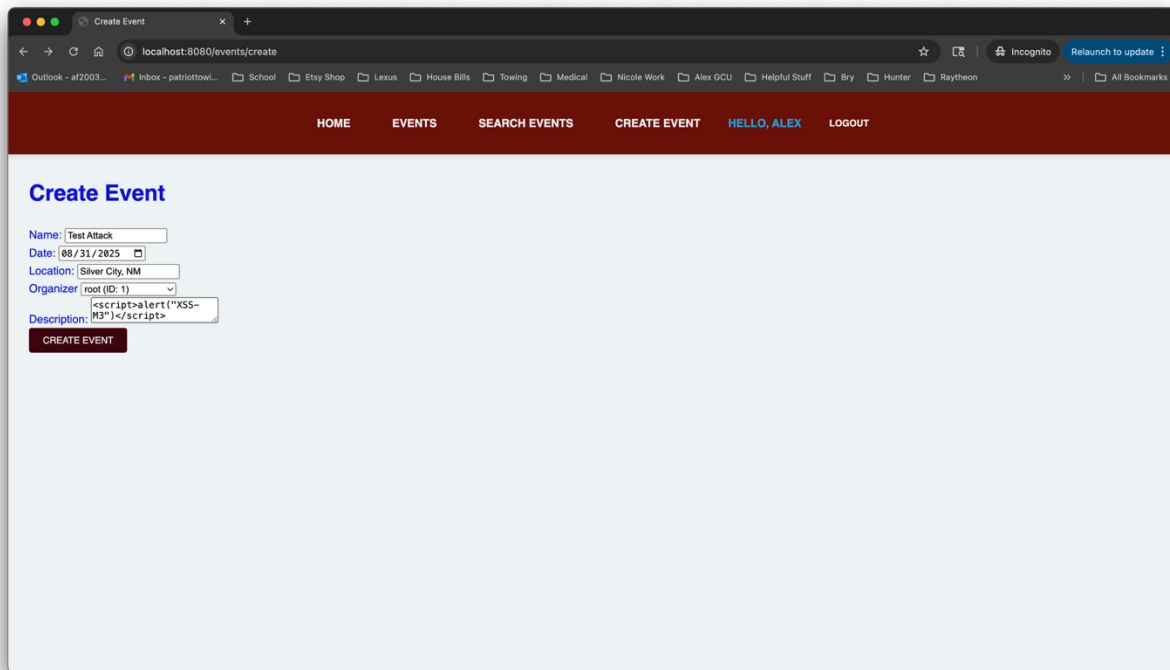


Figure 1 events.html uses th:utext to render event.description without escaping.

1.2 Proof-of-Concept Attack (repro steps)

Create Event With Payload



The screenshot shows a web browser window with the address bar displaying `localhost:8080/events/create`. The browser's bookmark bar includes links to Outlook, an inbox, School, Etsy Shop, Lexus, House Bills, Towing, Medical, Nicole Work, Alex GCU, Helpful Stuff, Bry, Hunter, Raytheon, and All Bookmarks. The website's navigation bar is dark red and contains links for HOME, EVENTS, SEARCH EVENTS, CREATE EVENT, HELLO, ALEX, and LOGOUT. The main content area is titled "Create Event" and contains a form with the following fields:

- Name:**
- Date:**
- Location:**
- Organizer:**
- Description:**

Below the Description field is a dark red button labeled "CREATE EVENT".

Figure 2 Creating an event with a malicious payload in the **Description** field.

XSS Popup On Events Page

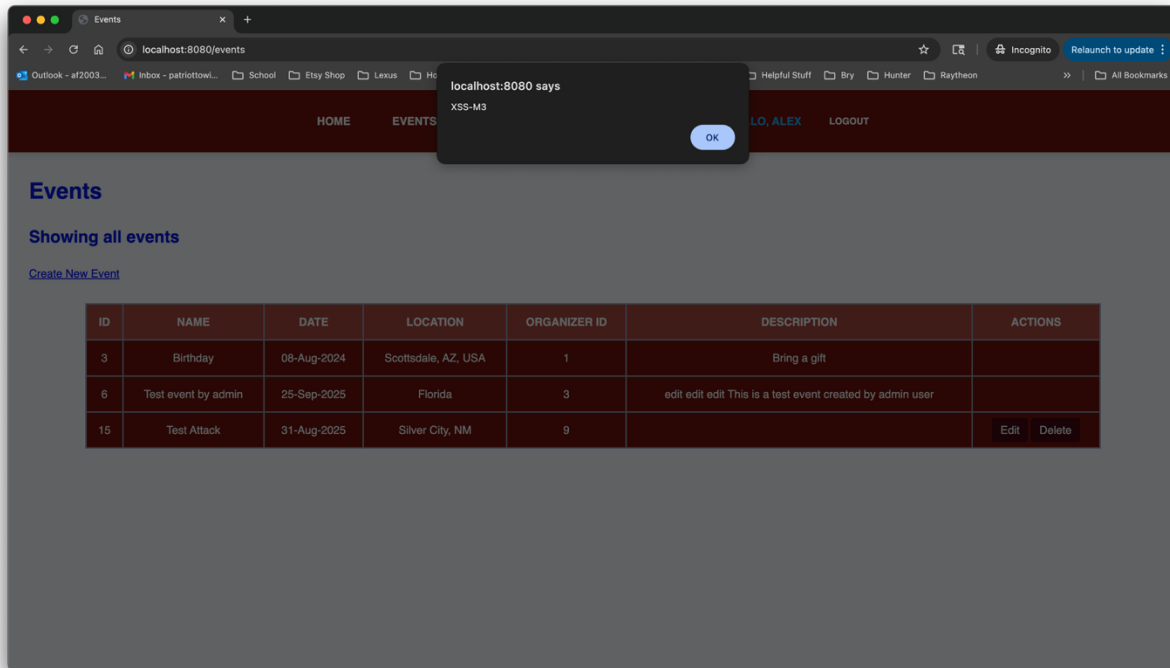


Figure 3 Stored XSS executes when the Events list renders the Description.

2.1 Fix the template sink (escape on render)

2.1 Fix the template sink (escape on render)



2.2 Add a sanitization library (jsoup)

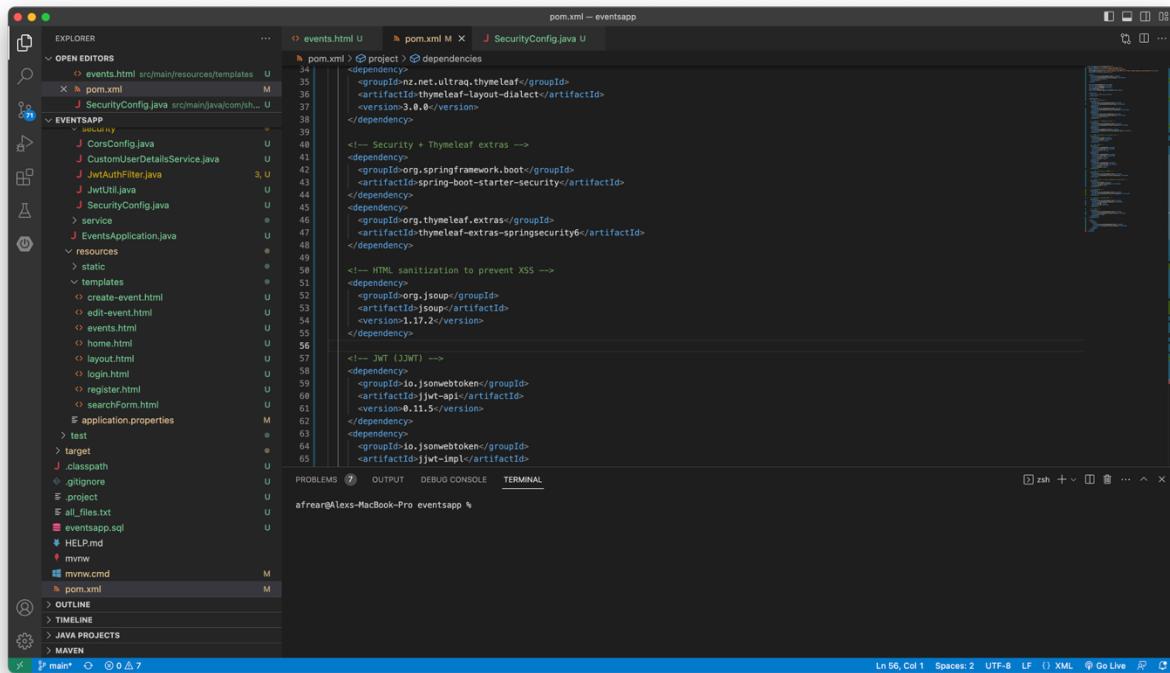


Figure 5 pom.xml updated with jsoup dependency for HTML sanitization.

2.3 Create a sanitizer component

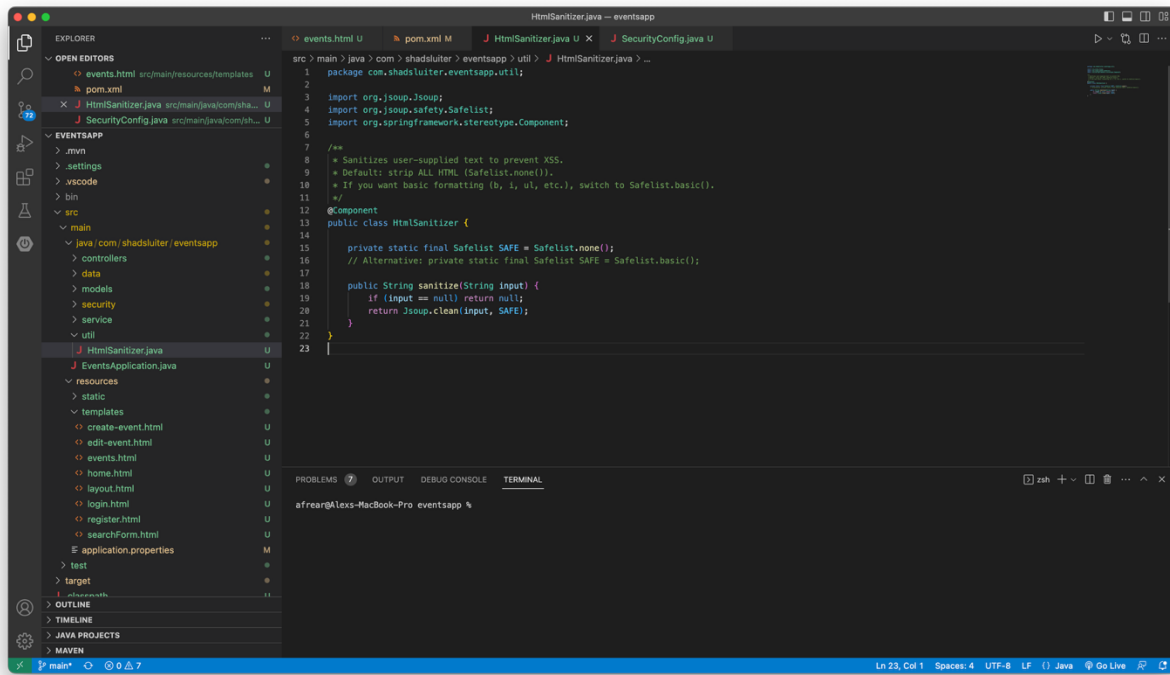


Figure 6 *HtmlSanitizer* Spring component strips unsafe HTML using jsoup.

2.4 Sanitize before saving

EventService Injection

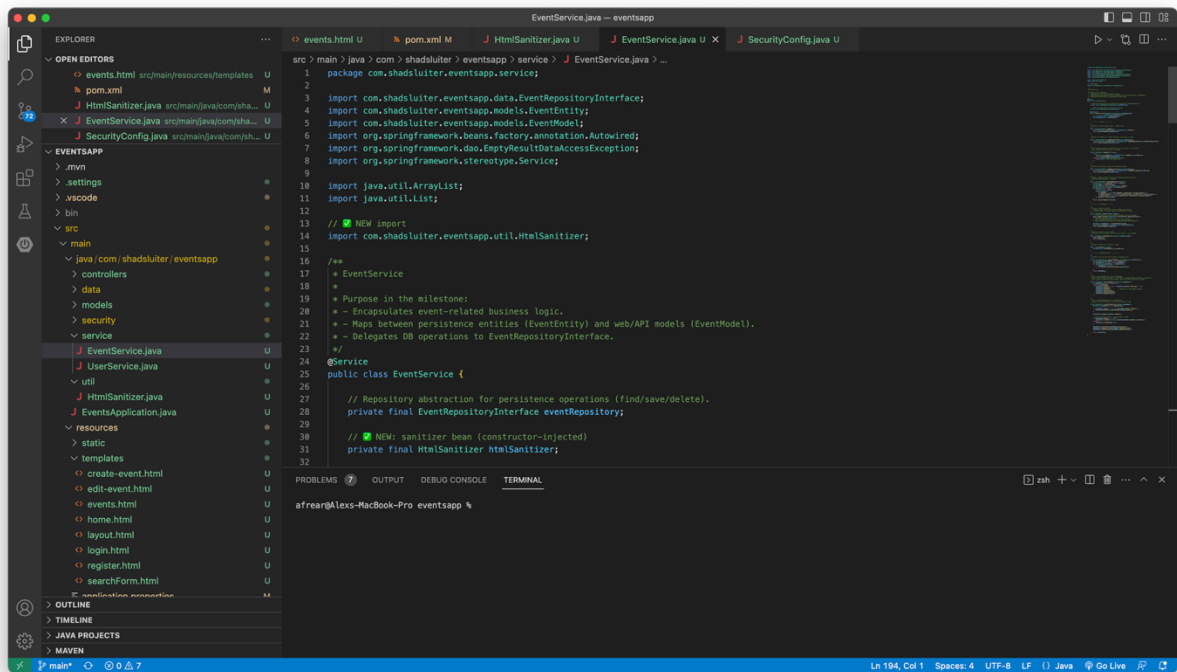


Figure 7 `EventService` now injects `HtmlSanitizer` via constructor.

EventService Save Sanitizes

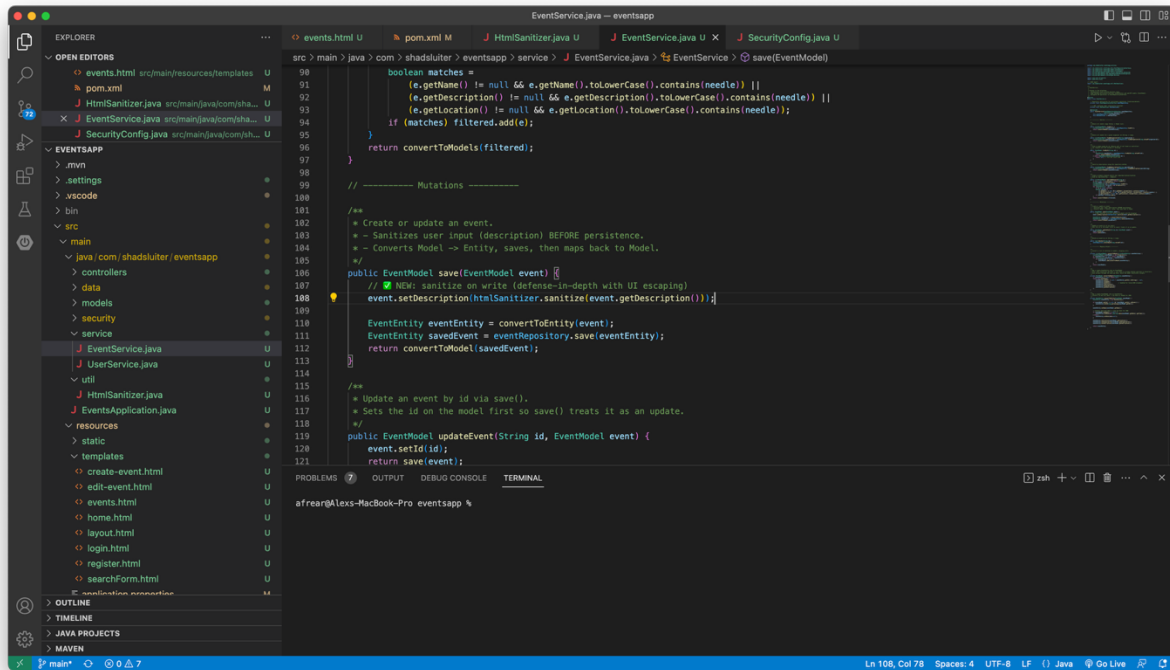


Figure 8 `save()` sanitizes `event.description` before persisting.

2.5 Rebuild & run

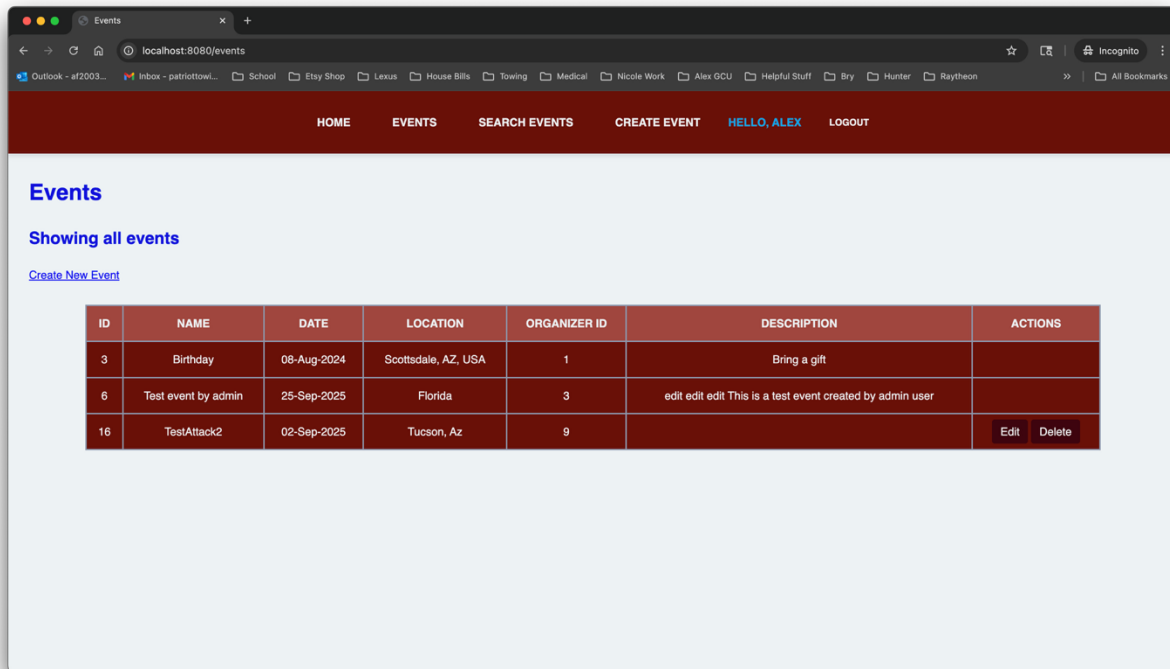


Figure 9 After applying data sanitization and escaping, the malicious `<script>` payload is stripped entirely. The description field is blank, confirming that the input was neutralized and did not execute.

Step 3 — Test the Application

3.1 Test Plan

Test Case	Input Description	Expected Result
1. Malicious Script	<code><script>alert("XSS-M3")</script></code>	No alert executes. Description field is blank (payload stripped).
2. Malicious Image Event	<code></code>	No alert executes. Description field is blank (payload stripped).
3. Normal Text Event	Company picnic at the park	Text is displayed as entered, safely escaped in HTML.
4. Mixed Input	Team lunch <code><script>alert("XSS")</script></code>	Only “Team lunch” is displayed. Script tag removed.

3.2 Execute the Tests

Malicious Script

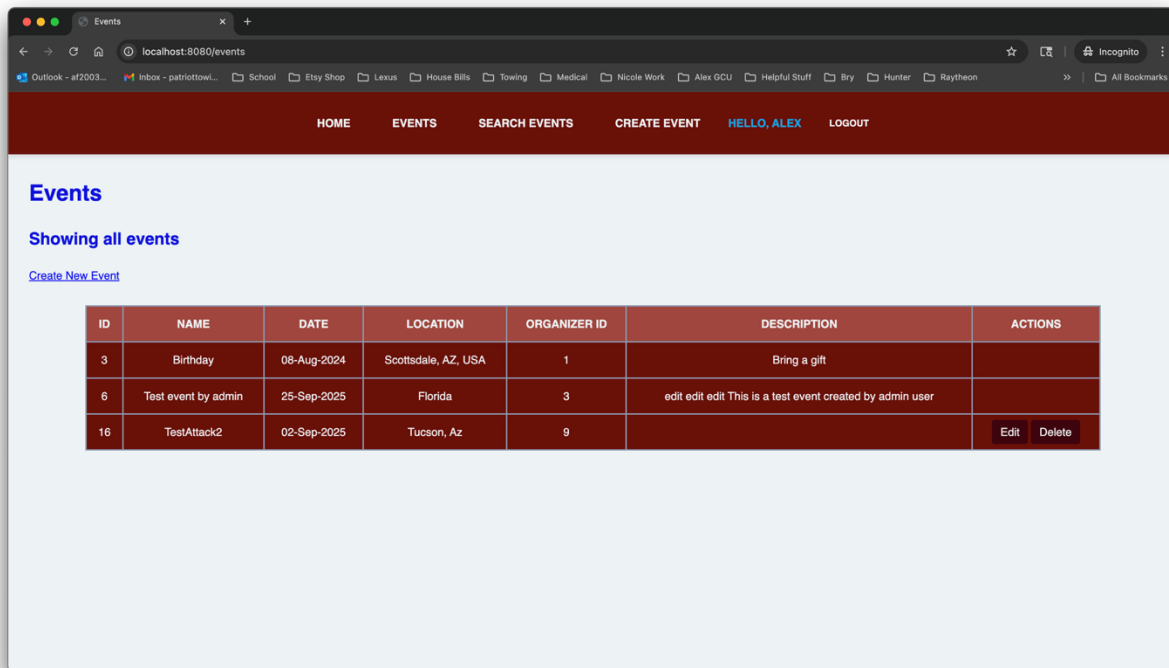


Figure 10 Malicious script input is stripped entirely; no execution occurs.

Malicious Image Event

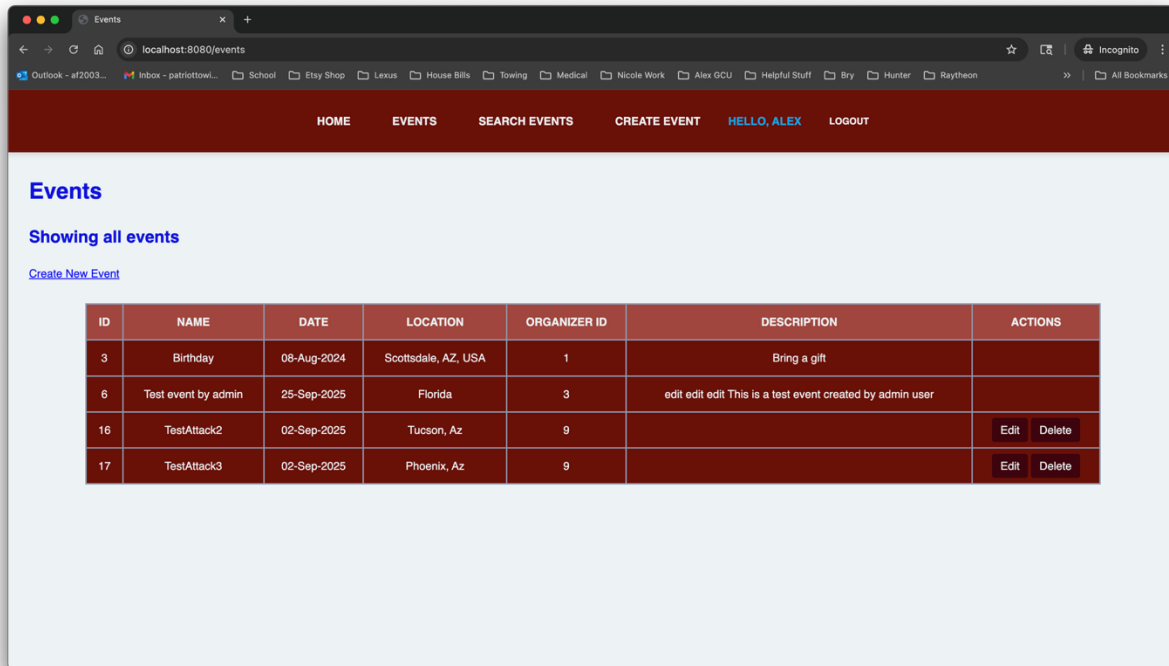
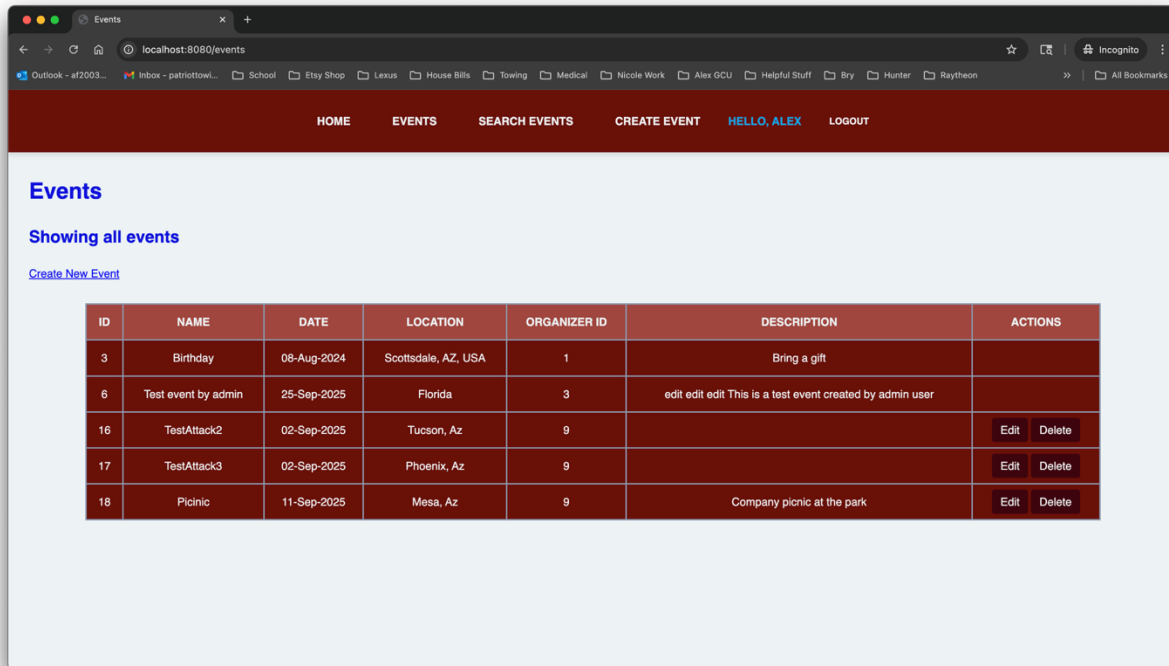


Figure 11 Image-based XSS payload stripped; no script executed.

Normal Text Event



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/events'. The browser's bookmark bar includes links to Outlook, an inbox, and various folders like School, Etsy Shop, Lexus, House Bills, Towing, Medical, Nicole Work, Alex GCU, Helpful Stuff, Bry, Hunter, and Raytheon. The application's navigation bar is dark red and contains links for HOME, EVENTS, SEARCH EVENTS, CREATE EVENT, HELLO, ALEX, and LOGOUT. The main content area is light blue and features the heading 'Events' in blue. Below this, it says 'Showing all events' and provides a link to 'Create New Event'. A table with seven columns (ID, NAME, DATE, LOCATION, ORGANIZER ID, DESCRIPTION, and ACTIONS) lists several events. The first event is a birthday on August 8, 2024. The second is a test event by admin on September 25, 2025. The next two are 'TestAttack2' and 'TestAttack3' on September 2, 2025. The final event is a picnic on September 11, 2025. The 'ACTIONS' column for the last three events contains 'Edit' and 'Delete' buttons.

ID	NAME	DATE	LOCATION	ORGANIZER ID	DESCRIPTION	ACTIONS
3	Birthday	08-Aug-2024	Scottsdale, AZ, USA	1	Bring a gift	
6	Test event by admin	25-Sep-2025	Florida	3	edit edit edit This is a test event created by admin user	
16	TestAttack2	02-Sep-2025	Tucson, Az	9		Edit Delete
17	TestAttack3	02-Sep-2025	Phoenix, Az	9		Edit Delete
18	Picnic	11-Sep-2025	Mesa, Az	9	Company picnic at the park	Edit Delete

Figure 12 Normal Text Event

Mixed Input

Events

Showing all events

[Create New Event](#)

ID	NAME	DATE	LOCATION	ORGANIZER ID	DESCRIPTION	ACTIONS
3	Birthday	08-Aug-2024	Scottsdale, AZ, USA	1	Bring a gift	
6	Test event by admin	25-Sep-2025	Florida	3	edit edit edit This is a test event created by admin user	
16	TestAttack2	02-Sep-2025	Tucson, Az	9		Edit Delete
17	TestAttack3	02-Sep-2025	Phoenix, Az	9		Edit Delete
18	Picnic	11-Sep-2025	Mesa, Az	9	Company picnic at the park	Edit Delete
19	TestAttackMixedInput	30-Sep-2025	Flagstaff, Az	9	Team lunch	Edit Delete

Figure 13 Normal text remains visible; malicious script portion stripped.

3.3 Final Validation

- No XSS payloads executed in any test.
- All malicious tags were either stripped or neutralized.
- Normal input displays correctly.
- The app is now resistant to stored XSS at the vulnerable point identified in Step 1.

Summary of Key Points

In this milestone, I found that the application was vulnerable to stored cross-site scripting because the events page was using `th:utext` to render the event description without escaping it. I was able to prove this by creating an event with a script tag in the description, which executed when the events list was loaded. To fix the issue, I updated the template to use `th:text` so the content is automatically escaped, added the `jsoup` library for sanitization, and created an `HtmlSanitizer` component to strip out unsafe HTML. I also modified the `EventService` class so the description is sanitized before it's saved to the database. After applying these changes, I tested the application with different inputs, including script tags, image tags with an `onerror` handler, and normal text. All of the malicious inputs were removed or stripped out, and no JavaScript executed. At the same time, safe inputs displayed normally. Overall, the application now properly sanitizes user input and is no longer vulnerable to the stored XSS issue that I identified at the beginning of the milestone.